

BuyWise Overview

Last updated: May 8, 2026

BuyWise helps shoppers check a product link before they buy. A user can paste a resale listing or a retail product page, and BuyWise gives a plain-English verdict on whether that link looks worth buying from, what risks to verify, what price to offer or target, and whether there are better used or retail options in the current BuyWise price guides.

The current product promise is:

Drop a product link. Know if it is the best place to buy.

BuyWise is designed for normal buyers, not professional resellers. The first users are people checking used or discounted laptops, cameras, bikes, and monitors from places like eBay, Craigslist, OfferUp, Facebook Marketplace, Best Buy, Amazon, or local sellers. The product is intentionally cautious: avoiding scams and bad purchases matters more than calling something a great deal too quickly.

What The Website Does

BuyWise currently supports these main flows:

- Paste a product or listing link on the homepage.
- Let BuyWise try to read the product title, price, source, and page summary from the link.
- Continue to a full verdict page when enough details are available.
- Add missing details manually if the site blocks link reading or hides the price.
- Search current BuyWise price guides.
- Save product guides and analyzed verdicts.
- Create an account, log in, and sync saved items across devices when Supabase is configured.
- View an About Us page that explains what BuyWise checks.

The website does not scrape Facebook Marketplace. Facebook Marketplace links are treated as resale links, but BuyWise asks the user to paste the listing title, price, and description instead of fetching the page.

Public Pages

Homepage

Route: `/\`

The homepage is the main product experience. The BuyWise logo and the Analyze Listing nav item both go here because link analysis is the core selling point.

The homepage has:

- A compact hero headline.
- A large link checker panel.
- Auto-fill indicators for product, price, and source.
- Resale listing and retail bargain modes.
- Optional extra details field.
- Resale and retail sample buttons.
- A "What you get back" section.
- Example product cards.
- A condensed example verdict section.

The homepage form asks for a product link first. It no longer asks users to manu

ally enter the product/model and price up front. When the user submits, the page stores a draft in session storage and sends the user to `/submit?draft=home`.

The homepage tries to extract link details before routing. If the link is readable, the submitted draft includes:

- Original URL.
- Inferred link type: resale or retail.
- Inferred marketplace/source.
- Extracted product title.
- Extracted price, if found.
- Extracted description or summary, if found.
- Closest BuyWise price guide match, if found.

If the link cannot be read, the draft still moves forward so the analyzer can ask for the missing details.

Analyzer Page

Route: `/submit`

The analyzer page handles both automatic and manual analysis.

If the user comes from the homepage and the draft has enough information, the page immediately builds the verdict and shows the result full width. The form is hidden after the verdict so the recommendation, reasoning, offer guidance, alternatives, questions, and checklist take over the screen.

If important details are missing, the page shows the form and asks for:

- Listing or product link.
- Product or model.
- Closest price guide.
- Link price.
- Link type: resale listing or retail bargain.
- Condition or deal state.
- Marketplace/source.
- Location.
- Listing text, retail page details, seller replies, or extra deal notes.

The analyzer gives:

- Verdict: Great deal, Fair price, Overpriced, Risky purchase, or Avoid.
- Deal score from 0 to 100.
- Risk level.
- Confidence score.
- Benchmark gap.
- Suggested offer for resale links.
- Target buy range for retail links.
- Red flags.
- Trust signals.
- Why this might not be worth it.
- Better retail moves.
- Better resale moves.
- Questions to ask.
- Meetup checklist or checkout checklist.
- Next steps before buying.

The analyzer uses deterministic local scoring from `lib/dealQuality.ts`. It does not use OpenAI yet.

Search Page

Route: `/search`

The search page lets users browse BuyWise price guides. Users can search by product name, category, brand, model, or year. They can also filter by category and minimum year.

Each result card includes:

- Product category.
- Brand and model.
- Year.
- Recommendation badge.
- Used average price.
- Fair price.
- MSRP drop.
- Reliability score.
- Demand score.
- Scam probability score.
- Short buyer guidance.
- Bookmark button.
- Link to the full product insight page.

The bookmark button saves the product. If the user is logged in and Supabase is configured, the save goes to the user's account. If not, it saves on the current device.

Product Insight Pages

Route: `/products/[id]`

Product pages explain a specific BuyWise price guide. Each page includes:

- Product name, category, year, and recommendation.
- Price summary.
- Depreciation chart.
- Common issues.
- Scam warnings.
- Buying checklist.
- Seller questions.
- Save listing form.
- Entry point back into the analyzer for that product.

Product pages are useful when a buyer wants context before checking a specific listing.

Saved Page

Route: `/saved`

The saved page shows saved products and saved verdicts. Each item can be tracked through statuses:

- Watching.
- Contacted.
- Negotiating.
- Bought.
- Passed.

Saved items work in two modes:

- Logged out or account sync unavailable: saved items are stored in browser local

l storage.

- Logged in with Supabase configured: saved items are stored in the Supabase `saved\_items` table.

When a user logs in, BuyWise syncs local saved items into the user's account and then clears the local copy to prevent duplicate local/account saves.

Account Page

Route: `/auth`

The account page supports sign up and log in through Supabase Auth.

It includes:

- Email field.
- Password field.
- Login/signup mode switch.
- Validation for valid email and minimum password length.
- Success and error states.
- Redirect to `/saved` when a login/signup returns an active session.

If Supabase environment variables are missing, the page shows a clean unavailable state and still lets the user use local-device saving.

About Us Page

Route: `/about`

The About Us page is customer-facing. It explains what BuyWise checks and why buyers should use it. It also shows credibility numbers from the current BuyWise price guide coverage:

- MSRP tracked.
- Used fair value tracked.
- Categories covered.
- Product guides.

Old `/admin` and `/monetization` URLs redirect to `/about`.

Link Reading

Server route: `/api/extract-link`

The link reader is a server-side route that tries to read safe public product pages. It exists so the homepage and analyzer can work from real URLs instead of only sample inputs.

The route does this:

1. Accepts a URL.
2. Normalizes missing protocols, such as `example.com/item`.
3. Rejects invalid URLs.
4. Rejects local/private/internal addresses.
5. Identifies the source label and source domain.
6. Infers whether the link is resale or retail from the domain.
7. Avoids fetching Facebook Marketplace.
8. Fetches public HTML with a timeout.
9. Reads only a limited amount of HTML.
10. Extracts product title from Open Graph, Twitter metadata, JSON-LD, or ``.11. Extracts description from metadata or JSON-LD.</div>

12. Extracts price from metadata, JSON-LD, embedded JSON, data attributes, or visible dollar amounts.
13. Matches the extracted title/description to the closest BuyWise price guide.
14. Returns structured data to the client.

The response can include:

- `ok`
- `manualRequired`
- `url`
- `sourceLabel`
- `sourceDomain`
- `mode`
- `marketplace`
- `title`
- `description`
- `price`
- `productName`
- `confidence`
- `message`
- `warnings`

This is real link reading, but it is not guaranteed for every website. Some sites block automated page reads, hide prices in scripts, require login, or return incomplete metadata. In those cases, BuyWise asks for manual details instead of pretending it knows more than it does.

Current Price Coverage

BuyWise currently has internal price guides for:

- Cameras.
- Laptops.
- Bikes.
- Monitors.

There are 20 product guides total:

- Cameras: Sony A6400, Canon EOS R10, Fujifilm X-T30 II, Sony A7 III, Canon M50 Mark II.
- Laptops: Apple MacBook Air M1, Apple MacBook Air M2, Dell XPS 13, Lenovo ThinkPad X1 Carbon, Microsoft Surface Laptop 5.
- Bikes: Trek Marlin 5, Specialized Rockhopper, Giant Talon, Cannondale Quick, Trek FX 3.
- Monitors: Dell UltraSharp U2720Q, LG 27GL850, ASUS ProArt PA278QV, Samsung Odyssey G5, BenQ PD2700U.

Each product guide includes:

- MSRP.
- Used low price.
- Used average price.
- Used high price.
- Fair price.
- Depreciation percent.
- Reliability score.
- Demand score.
- Scam probability score.
- Common issues.
- Best years/models.
- Models to avoid.
- Buying checklist.

- Seller questions.
- Recommendation label and explanation.

When a pasted link matches one of these guides, BuyWise can produce a stronger verdict. When a product is not covered yet, BuyWise asks the user to choose the closest guide instead of forcing a weak match.

Deal Scoring

Core file: `lib/dealQuality.ts`

The deal scoring logic uses structured inputs:

- Asking price.
- Fair price benchmark.
- Used low/high range.
- Reliability score.
- Scam probability score.
- Condition.
- Marketplace/source.
- Listing text.

It returns:

- Deal score.
- Recommendation.
- Explanation.
- Suggested offer low/high.
- Risk level.
- Price difference.
- Confidence score.
- Red flags.
- Positive signals.
- Negotiation tip.
- Next steps.

The recommendation can become stricter when the listing has risk signals. For example, a cheap used listing can still become "Risky purchase" if the seller wording is weak, the proof is missing, or the payment/meetup terms look unsafe.

Red flag detection looks for wording such as:

- Must go today.
- Cash only with rushed tone.
- Shipping only from a local listing.
- No returns.
- Stock photos.
- No receipt.
- Locked account/device language.
- Too-good-to-be-true pricing.
- Vague condition claims.

Positive signals include details such as:

- Original receipt.
- Serial number.
- Warranty.
- Working video.
- Public meetup.
- Original box.
- Battery health.
- Accessories included.

Confidence is higher when the pasted details include proof, condition, price context, and seller specifics. Confidence is lower when the link or pasted text is vague.

Retail Versus Resale Logic

BuyWise supports two analysis modes.

Resale listing mode compares the link price against used fair value. It focuses on scam risk, condition, seller proof, meetup safety, and negotiation.

Retail bargain mode compares the sale price against MSRP and used alternatives. It focuses on whether the retail sale is actually meaningful after considering warranty, return policy, refurbished/open-box status, and used-market prices.

The goal is not just "used is cheaper" or "new is safer." The goal is to identify the better place to buy for the specific product and price.

Alternatives

Core file: `lib/linkAnalysis.ts`

BuyWise shows alternatives when the current price guides contain a clearly better move.

Better retail moves can include:

- A new retail benchmark that is close enough to make warranty/returns worth considering.
- A retail MSRP comparison when the used listing is overpriced.
- A retail fallback when a resale link is risky.

Better resale moves can include:

- A fair used target price.
- A stronger used alternative in the same category.
- A reason to keep watching instead of buying the pasted link.

If no better move exists in the current BuyWise price guides, the UI says that clearly.

Saving Behavior

Product cards and verdict cards both support saving.

Logged out behavior:

- Saves are stored in browser local storage.
- The saved page works on the same device.
- The user can still analyze links and track items.

Logged in behavior:

- Saves are inserted into Supabase `saved\_items`.
- Saved items are tied to the authenticated user id.
- Local saved items sync into the account after login.
- Local synced copies are cleared after successful sync.

Analyzed listing checks are also stored in Supabase `listing\_checks` when the user is logged in.

Supabase Setup

Supabase is used for:

- User accounts.
- User profiles.
- Saved items.
- Listing check history.

Required environment variables:

```
```bash
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
```
```

These belong in ``.env.local`` locally and in Vercel environment variables for the live site.

The schema is in:

```
`supabase/schema.sql`
```

The schema creates:

- ``profiles``
- ``products``
- ``product_issues``
- ``buying_checklist_items``
- ``seller_questions``
- ``saved_items``
- ``listing_checks``

It also:

- Enables row level security.
- Adds owner-only policies for profiles and saved items.
- Adds listing check policies.
- Adds a trigger that creates or updates a profile when a Supabase Auth user is created.
- Adds indexes for saved item and listing check queries.

The saved item and listing check tables do not require product ids to exist in the Supabase ``products`` table, because the app currently reads price-guide content from the local product data file.

Local Storage

BuyWise uses browser storage in two places:

- ``buywise.listingDraft.v1`` in session storage for homepage-to-analyzer drafts.
- ``buywise.saved-items`` in local storage for saved items when the user is not logged in.

This lets the website keep working even before account sync is available.

Navigation

The header contains:

- BuyWise logo on the left.
- Search, Analyze Listing, and Saved centered on desktop.

- About Us near the right side.
- Account control on the far right. Logged-out users see Log In. Logged-in users see their email and a Log out button.

The footer contains:

- A short product sentence.
- Search.
- Analyze Listing.
- Saved.
- About Us.
- Account.

There is no public admin page and no public build-planning page. Old `/admin` and `/monetization` URLs redirect to `/about`.

Key Files

- `app/page.tsx` - Homepage.
- `components/HomeListingPrompt.tsx` - Homepage link checker.
- `app/submit/page.tsx` - Analyzer route.
- `components/ListingAnalyzerForm.tsx` - Main analyzer form and auto-analysis behavior.
- `components/DealScoreCard.tsx` - Verdict/result UI.
- `app/api/extract-link/route.ts` - Server-side link reader.
- `lib/dealQuality.ts` - Core scoring, risk, confidence, offer, and next-step logic.
- `lib/linkAnalysis.ts` - Link mode/source inference and alternative suggestions.
- `lib/productMatch.ts` - Product matching against current price guides.
- `data/mockProducts.ts` - Current internal starter product data.
- `components/ProductCard.tsx` - Product guide cards and bookmark saves.
- `components/SavedItemsClient.tsx` - Saved item loading, syncing, status updates, and deleting.
- `components/AuthForm.tsx` - Login/signup form.
- `lib/supabaseClient.ts` - Supabase client setup.
- `lib/savedItemsStorage.ts` - Local saved item fallback.
- `supabase/schema.sql` - Database schema.

What Works Now

- Homepage link-first flow.
- Real public link metadata/price extraction when the page exposes readable data.
- Resale and retail analysis modes.
- Full-screen verdict flow from homepage drafts.
- Manual analyzer fallback when data is missing.
- Product search and product guide pages.
- Product card bookmarking.
- Verdict saving.
- Local saved items.
- Supabase account signup/login when env vars and schema are configured.
- Supabase saved item sync when logged in.
- Listing check history writes when logged in.
- Public UI cleanup: no visible admin, setup, or build-planning copy.

Practical Limits

BuyWise can only give strong benchmark-based verdicts for products covered by the current price guides. For unsupported products, it can read the link but still needs the user to pick the closest available guide or wait until that product category is added.

Some websites block link reading. BuyWise handles that by asking for the missing details manually.

Facebook Marketplace is not fetched. Users should paste the listing text and price.

The current analyzer uses local deterministic logic, not an OpenAI API. That keeps the product fast and cost-free while the user experience is being validated.

Manual Setup Needed For Production Accounts

For accounts and cloud saves to work on the live site:

1. Create or open a Supabase project.
2. Run `supabase/schema.sql` in the Supabase SQL editor.
3. Add `NEXT\_PUBLIC\_SUPABASE\_URL` and `NEXT\_PUBLIC\_SUPABASE\_ANON\_KEY` to `.env.local` for local development.
4. Add the same variables to the Vercel project environment variables.
5. Redeploy the Vercel site after adding the variables.
6. Test sign up, log in, product save, verdict save, saved item status changes, and deletion.

Deployment

The connected GitHub repository is:

`https://github.com/LukeShew/BuyWise.git`

The live site is:

`https://trybuywise.com`

The local site should run on:

`http://localhost:3000/`

Local checks:

```
```bash
npm run typecheck
npm run lint
```